

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: PROYEC-20 — Validador de Prompts y Tokens

1. CONTEXTO

El equipo SDET requiere una utilidad interna de baja fricción para estimar tokens y latencia de prompts antes de integrarlos al orquestador principal. El requerimiento impone explícitamente una restricción de entrega de máximo 2 archivos (Single-File Architecture). No hay usuarios concurrentes masivos, no hay persistencia de datos, no hay autenticación, y el backend es una simulación matemática pura sin conexión a LLMs reales.

2. DECISIÓN ARQUITECTÓNICA

****Arquitectura elegida: Dual-File Static + Micro-API (Single-Process)****

- `index.html` !' Frontend completo (HTML + CSS via Tailwind CDN + JavaScript vanilla con fetch API)
- `main.py` !' Backend FastAPI con un único endpoint POST `/api/analyze`, CORS habilitado para localhost, simulación asíncrona con `asyncio.sleep()`

No se utiliza ningún framework frontend (React, Vue, Svelte), ningún bundler (Webpack, Vite), ningún gestor de estado, ninguna base de datos, ningún ORM, ningún sistema de colas y ningún contenedor Docker. El stack se reduce a: ****Python 3.x + FastAPI + Uvicorn**** en backend, y ****HTML5 nativo + Tailwind CDN**** en frontend.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS

- ****¿Por qué NO microservicios?**** Un solo desarrollador puede desplegar ambos archivos. No hay equipos independientes. No hay dominios de negocio separados. Agregar microservicios sería ingeniería especulativa pura.
- ****¿Por qué NO React/Vue?**** La UI tiene exactamente 3 elementos interactivos: un textarea, un botón y un panel de resultados. Introducir un framework SPA añadiría ~200KB de dependencias, un proceso de build y una curva de onboarding para cero beneficio funcional.
- ****¿Por qué NO base de datos?**** El requerimiento no contempla persistencia histórica de análisis. PostgreSQL, SQLite o cualquier motor de almacenamiento violaría YAGNI en este contexto.
- ****¿Por qué NO Pydantic models complejos?**** Un único modelo de request/response es suficiente. La validación de campo vacío ocurre en el frontend antes de la petición de red.
- ****¿Por qué NO Docker/docker-compose?**** La herramienta es interna, de uso local. El overhead de containerización no aporta valor para este MVP.
- ****¿Por qué FastAPI y no Flask/Django?**** FastAPI provee soporte nativo `async/await`, validación automática con Pydantic, documentación Swagger auto-generada en `/docs`, y CORS middleware integrado. Es la elección 'boring technology' más adecuada para un micro-backend Python en 2024.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (Bajo):**** Si en el futuro se requiere persistencia de análisis históricos, se deberá refactorizar `main.py` para agregar SQLite o PostgreSQL. Costo estimado: 2-4 horas de trabajo.
- ****Riesgo 2 (Bajo):**** La estimación de tokens (palabras * 1.3) es una aproximación matemática, no un tokenizador real (tiktoken, sentencepiece). Si se requiere precisión real, se deberá integrar la librería correspondiente al modelo objetivo.
- ****Riesgo 3 (Muy Bajo):**** Tailwind via CDN genera un bundle completo (~3MB). Para producción real se recomendaría purgar clases no usadas. Para uso interno local, el impacto es despreciable.
- ****Riesgo 4 (Bajo):**** Sin autenticación ni rate-limiting. Aceptable para herramienta interna en red local. Inaceptable si se expone a internet.

5. CONTRATO DE INTERFACES

- **Puerto Backend:** `8000` (Uvicorn default)
- **Endpoint:** `POST http://localhost:8000/api/analyze`
- **Request Payload:** `{ "prompt": "string" }`
- **Response Payload:** `{ "tokens\_estimados": int, "latencia\_ms": int, "estado": "Óptimo" | "Alerta" }`
- **CORS Origins permitidos:** `[ "http://localhost", "http://127.0.0.1", "null" ]` (el origen `null` cubre archivos HTML abiertos directamente desde el sistema de archivos via `file://`)
- **Regla de negocio tokens:** `floor(len(prompt.split()) \* 1.3)`
- **Regla de negocio estado:** `"Alerta"` si `tokens\_estimados > 1000`, `"Óptimo"` en caso contrario
- **Latencia simulada:** `random.randint(500, 2000)` ms via `asyncio.sleep(latencia\_ms / 1000)`
- **Comando de arranque:** `uvicorn main:app --reload --port 8000`

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 23,156

Tokens de salida (output): 7,324

Total tokens: 30,480

Horas-hombre estimadas: ~2.5 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,147 input | !'1,466 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"4,968 input | !'3,000 output | 2 llamadas

FRONTEND-DESARROLLO: !"7,064 input | !'1,875 output | 2 llamadas

QA-VALIDACION: !"8,977 input | !'983 output | 2 llamadas